Hyderabad Property Price Prediction

Exploration, Feature Engineering and Modelling: what we did, how we did it, and what came out

Phases 2 and 3 of 4 | Source: MagicBricks listings, scraped into SQLite | 27 October 2026

1. What this report covers

The goal of the project is a model that predicts the **asking price** of a residential property in Hyderabad from the things you can see in a listing: how big it is, how many bedrooms and bathrooms it has, what type of building it is, and where it is.

An earlier report covered cleaning the scraped data. This one covers everything after that: looking at the cleaned data to decide what to do with it, turning it into something a model can read, training four models, and measuring how well they actually work on listings they had never seen.

The language here is deliberately plain. Where a technical term is unavoidable it is explained the first time it appears.

The headline result

The best model is **Gradient Boosting**. On 3,868 held-out listings it was wrong by **Rs 53 lakh** on a typical property, which is **16%** of the price, and **66% of its predictions landed within 25%** of the true asking price.

Measure	Result	What it means in plain words
Listings used for training	15,472	80% of the cleaned data
Listings held back for testing	3,868	never used to make any decision
Features given to the model	23	out of 71 available columns
Typical error (MAE)	Rs 53 lakh	the average size of a miss
Typical error as a %	16%	the same miss, relative to price
Predictions within 25%	66%	roughly how often it is useful
R-squared (log price)	0.78	share of the variation explained
Beats 'always guess the median' by	54 lakh	proof the model learned something

For context on what counts as good: predicting house prices from listing fields alone is hard, because two flats of the same size on the same street can legitimately differ by 30% on floor, view, finish and how badly the owner wants to sell. None of that is in the data. An error around 16% is a reasonable baseline result and the error analysis in section 7 shows exactly where it comes from.

2. The data we started with

The cleaning phase left **19,340 listings and 71 columns**, combining two tables: one row per property, joined to a table of locality-level ratings and published price statistics. Everything in this report starts from that file.

One housekeeping note, so the two reports can be read side by side: the cleaning report quotes 19,715 rows, while the file in `data/processed/` that this work actually used holds 19,340. The difference is a later revision of the same cleaning filters that was not written back to this folder. Every number in this report comes from the file that was really used, and re-running both notebooks against the newer file would move the results only marginally — it is 2% more data through an identical pipeline.

One inherited caveat matters throughout. The data was scraped price band by price band (1–50 lakh, 50–80 lakh, 80 lakh–1 crore, and upward), so the mix of cheap and expensive listings reflects how many pages each

band had, not the real Hyderabad market. That is fine for a model that answers “given these features, what price?”, which is our question. It is not a basis for claims about the market as a whole.

3. First move: split the data before looking at it

Before computing a single statistic we set aside 20% of the listings and did not look at them again until the very last step.

The reason is easy to miss. Exploring data is not a passive activity: we were about to decide which columns to keep, what to fill missing values with, and which categories to merge. Every one of those decisions is based on what we saw. If we had looked at all the data first, those decisions would have quietly carried information from the test listings into the model, and the final score would have flattered it.

So the rule for the whole project was: **every number we looked at came from the training set only**. 15,472 listings to learn from, 3,868 kept sealed.

4. What we found when we explored (Notebook 03)

We limited the exploration to six questions. Each one had to lead to a decision; anything that would only have produced a nice-looking chart was skipped.

4.1 The price needed to be rescaled

Asking price is extremely lopsided. The middle listing is about Rs 1.5 crore, but the most expensive is Rs 49 crore, so a thin tail of very expensive property stretches far to the right. Measured formally, the skew was **5.82**, where anything above about 1 is considered lopsided.

Why this matters: Linear and Ridge Regression work by minimising *squared* error, so a handful of enormous prices dominate everything the model is trying to do. It ends up fitting mansions and ignoring the flats that make up most of the data.

The fix is to train on the **logarithm** of the price instead. A log compresses large numbers much more than small ones, which pulls the long tail in. It took the skew from 5.82 to **0.09** — essentially symmetrical.

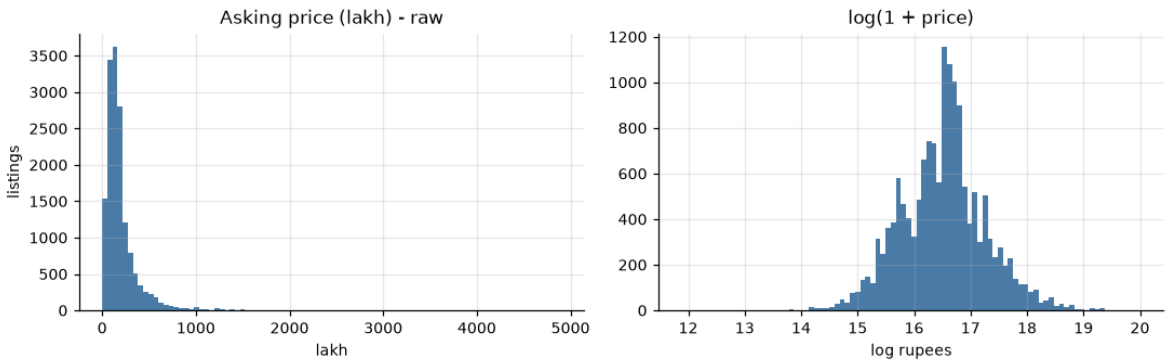


Figure 1. Asking price before and after taking logs. The left chart is almost unusable for a linear model; the right one is close to a bell curve.

Every model in this project therefore predicts log price, and every prediction is converted back into rupees before we score it — so the errors we report are in rupees, which a human can actually judge.

4.2 Area is the strongest honest feature — and one column was cheating

Ranking the numeric columns by how strongly they move with log price put covered area at the top, followed by bathrooms, bedrooms and the locality's published price per square foot.

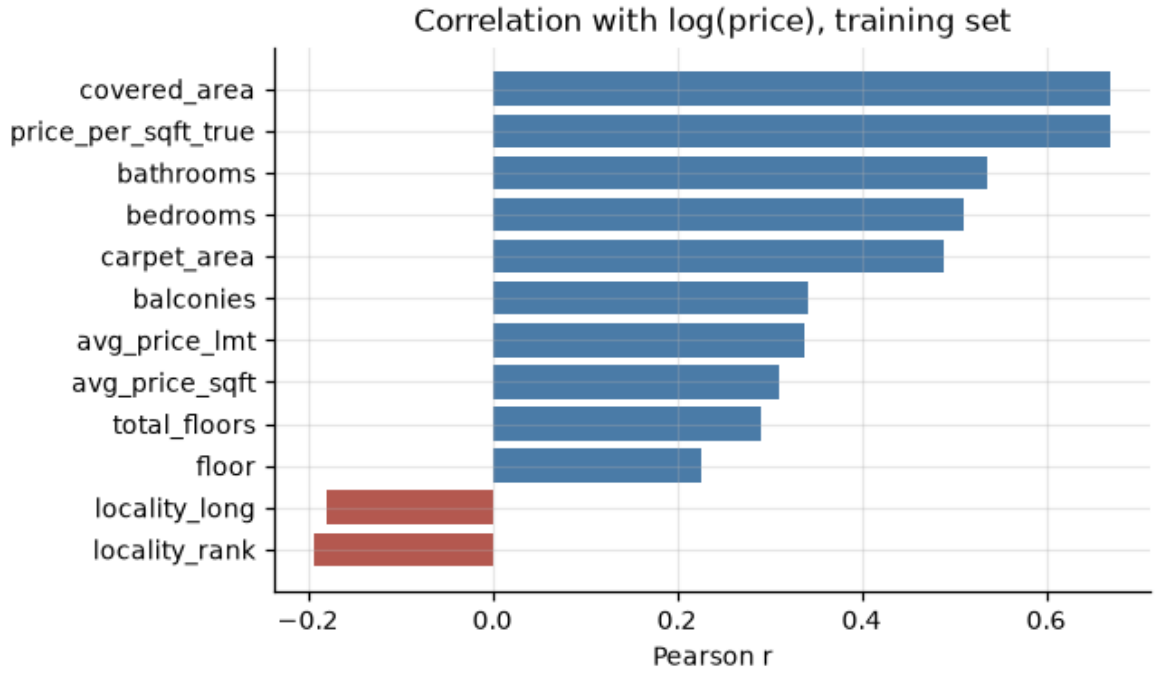


Figure 2. How strongly each number moves with log price. Negative is not weaker: locality_rank is negative because rank 1 is the best locality.

Tied for first place, though, was a column called **price_per_sqft_true** — and that one had to go. It is the price divided by the area, so it contains the answer. We proved it rather than assumed it: dividing price-by-area by that column gives exactly 1.0 on all 15,472 training rows.

This is called **leakage**, and it is the single most common way a beginner project ends up with a model that scores beautifully and is worthless. Left in, it would have pushed R-squared above 0.95. The test is always the same question: *would I know this value before the price?* For price per square foot, obviously not — you would need the price.

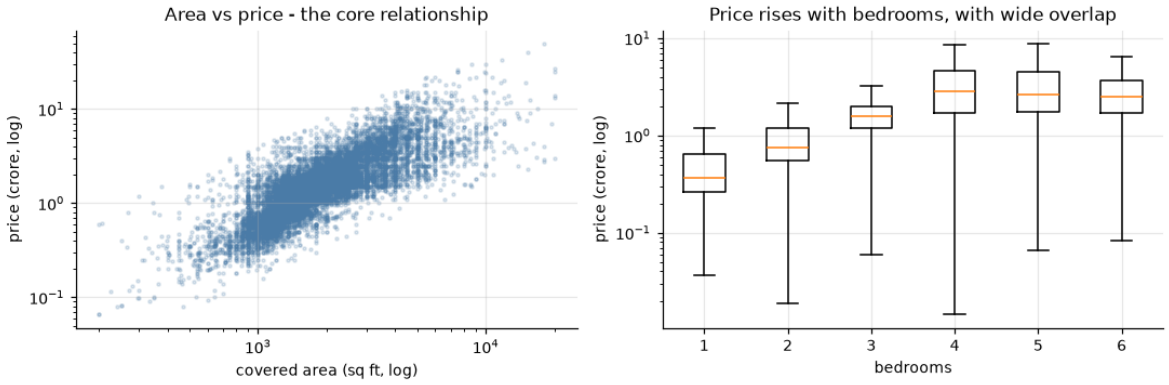


Figure 3. Left: area against price, both on log scales, is nearly a straight line. Right: price rises with bedrooms, but the boxes overlap heavily, which is why location features matter.

4.3 Missing values, grouped by why they are missing

60 of the 71 columns had gaps. The useful move was to ask *why* each one was empty instead of applying one blanket rule.

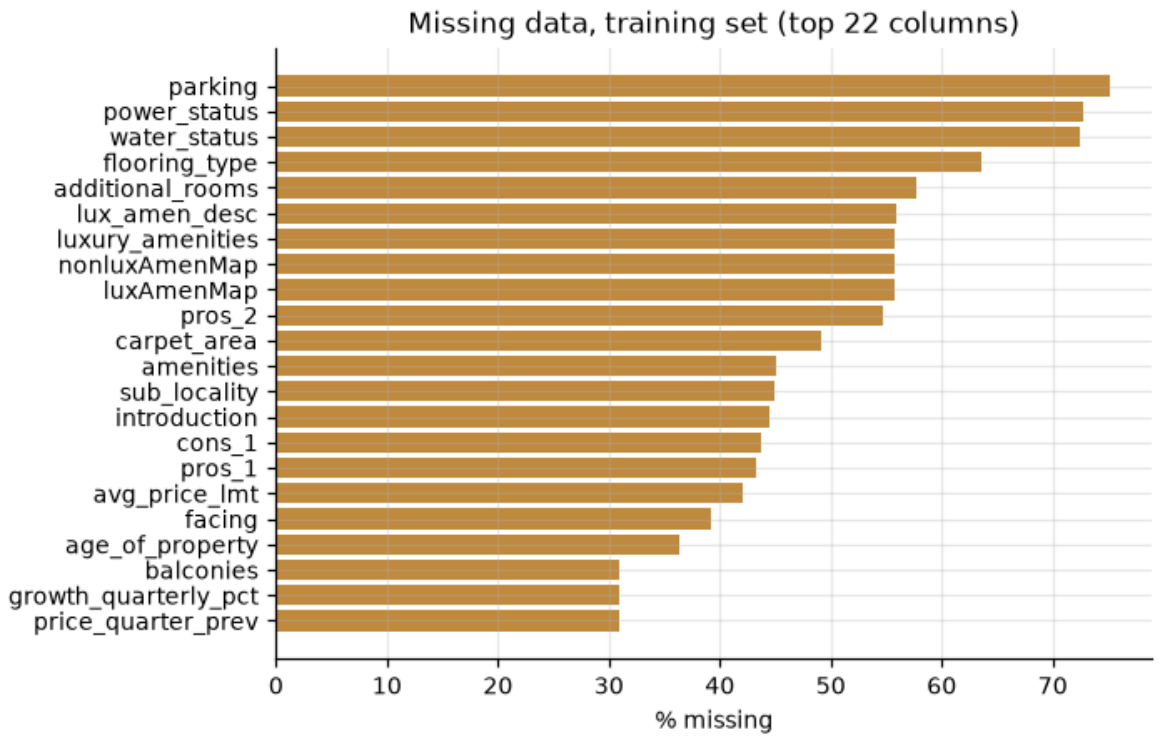


Figure 4. The emptiest columns in the training data.

The best example is the floor number, missing on 28% of listings. That looks like a data quality problem until you break it down by property type:

Property type	Floor number missing
Villa	99.9%
Residential House	50.6%
Apartment	4.6%
Builder Floor Apartment	5.0%

It is not a quality problem at all. “Which floor?” is a meaningless question for a standalone villa, so the field is blank. **The blank itself is information** — it tells you the building type. So rather than just filling the gap, we fill it *and* add a small extra column recording that it was filled. The model can then learn from the fact of the gap, which is exactly what we want.

The same reasoning decided the rest: **facing** and **age of property** became an explicit “Unknown” category rather than a guess; **carpet area** was dropped, because it is half empty and duplicates covered area, which is better filled and more predictive; and the amenity detail fields (parking, power, water, flooring), 64–75% empty, were set aside for this first version.

4.4 Two columns were broken and needed repair, not deletion

facing claimed 34 different values. Only eight of them are compass directions. The rest were whatever sellers typed into the wrong box, such as *“Near Nallakunta PS. Beside Jyothi Bala Mandir High School”*. One-hot encoding it as-is would have created a separate column for each landmark, every one matching a single property, every one pure noise.

But the 255 odd values were not all the same problem, and that distinction was worth making:

Value	Rows	What it is	What we did
Newly Constructed Property	176	an answer to a different question	→ Unknown
“South -West”	54	a real direction missing a space	→ fixed the spacing
landmark descriptions	~25	free text in the wrong field	→ Unknown

Those 54 rows are the point. A careless clean-up would have thrown away 54 perfectly good observations because of one missing space.

possession had 48 values, but 13,830 of them said “Ready to Move” and 1,111 said “Under Construction”. The rest were individual handover dates, plus “Ready To Move” with a capital T, which a computer treats as a completely different category. We collapsed all of it into three sensible levels, which fixes the capital-letter bug for free.

4.5 Location: the hardest decision in the project

Location drives property price more than anything else, and it was the hardest thing to encode. The training data contains **2,177 different localities** for 15,472 listings — about seven listings each. Only 129 of them have 20 listings or more.

The standard beginner approach, one-hot encoding, would create 2,177 almost-empty columns, and any locality appearing once in training is a column that teaches the model nothing. Project and builder names were worse: 8,876 and 10,388 distinct values, nearly all unique. Those are names, not categories.

The alternative was already sitting in the data. The locality table carries numbers the website publishes per area — average price per square foot, a locality rank, quarterly price growth. Each one is **a single column that works for every locality**, including localities the model has never seen.

Two things make these safe to use, and both are worth stating precisely:

- They are published *before* any individual listing exists, so they would genuinely be available at prediction time. That is the real test for leakage.
- They are **not** target encoding. Target encoding means computing the average price of *our own* training listings per locality, which leaks the answer and must be rebuilt inside each validation fold. These come from the website, computed over the whole market.

We did not take the argument on trust — section 6.2 tests it.

5. Turning the data into something a model can read (Notebook 04)

This is the step people usually overcomplicate. We kept it deliberately small, because a correct target transform and an honest testing protocol buy far more than invented features do, and every extra transformation is another place for a bug to hide.

In total: **23 features** (14 numeric, 9 categorical) out of 71 columns. Two columns repaired. 49 dropped. No new mathematical features, no interactions, no clustering, no text mining.

5.1 What was dropped, and why

Group	Columns	Reason
Leakage	1	price_per_sqft_true is the price divided by the area
Identifiers	5	property id, project and builder names — near-unique text

Group	Columns	Reason
Free text	11	descriptions state the price in words in 86% of listings
Redundant	11	duplicate coordinates, inputs of a column we already keep, raw versions of repaired columns
Too empty / needs parsing	5	amenity detail fields, 64–75% empty; flooring needs string splitting
Individual ratings	16	16 locality ratings that all move together; we keep the overall one

The free-text group deserves a note, because it is the subtlest finding inherited from the cleaning phase: MagicBricks auto-generates the description, and 86% of those descriptions state the price in words — *"this apartment is sold at price of Rs 46.9 Lac"*. Any text feature built on them would have carried the answer. Leakage is a property of the information, not of the data type.

5.2 The recipe that learns from training data only

All the preparation sits inside a scikit-learn **Pipeline**, which is best understood as a sealed recipe: it learns its numbers from whatever data you fit it on, and then applies exactly those numbers to anything you give it afterwards. It does three jobs.

Step	What it does	Why
Fill numeric gaps	use the middle value (median), and add a 0/1 marker column recording that it was filled	median because the data is lopsided; the marker column becau
Put numbers on one scale	rescale so each column has a similar spread	Ridge penalises large coefficients, so without this the penalty would
Encode categories	one column per category (one-hot), rare categories pooled together	models need numbers, not words; pooling categories with under 10 l

Two consequences are worth spelling out, because they are the whole reason for using a pipeline rather than preparing the data by hand:

- When we cross-validate, the median and the category lists are **recomputed from scratch on each fold**. Had we computed the median once over the whole training set, every fold would have been scored with a number derived partly from the rows it was being tested on.
- The same recipe object is used for all four models, so the comparison between them is fair: the only thing that changes is the model.

The 23 features expand to **67 columns** after one-hot encoding and the missing-value markers.

6. How we trained and compared the models

6.1 The protocol, in plain words

- **Cross-validation.** Split the training data into five parts; train on four, predict the fifth; repeat five times so every listing gets predicted by a model that never saw it. This is how all model choices were made, and it is far more trustworthy than a single split.
- **Four metrics.** *MAE* is the typical size of a miss, in lakh — the headline. *RMSE* punishes big misses harder, so RMSE far above MAE means a few predictions are badly wrong. *R-squared* is the share of the variation explained, on the log scale the model was fitted on. *MedAPE* is the typical miss as a percentage, which MAE cannot see — Rs 20 lakh is a disaster on a Rs 50 lakh flat and a rounding error on a Rs 10 crore villa.
- **A stupid baseline first.** Before trusting any model, we scored the dumbest rule there is: always predict the median price. Anything that cannot beat it by a wide margin has learned nothing.
- **The test set was opened once**, at the end, after every choice had been made.

6.2 Five-fold cross-validation on the training set

Model	MAE (lakh)	RMSE (lakh)	R-squared (log)	Typical error %
Baseline (median)	108.1	216.4	-0.000	40.0%
Linear Regression	76.7	273.6	0.633	25.3%
Ridge Regression	76.7	273.6	0.633	25.3%
Random Forest	52.8	131.4	0.773	16.4%
Gradient Boosting	56.5	136.0	0.758	18.5%

The ranking is clear and the reason is worth understanding, because it is the most instructive result in the project. **Both tree models beat both linear models by a wide margin.**

A linear model has to assume that an extra square foot adds the same number of rupees everywhere in Hyderabad. It plainly does not — a square foot in Jubilee Hills and one on the outskirts are different goods. That dependence of one feature's effect on another is called an **interaction**, and a linear model cannot represent it unless you build the combined term by hand. A tree splits on the locality's price level first and then on area inside each branch, so it discovers the structure on its own.

Linear and Ridge landing close together is also informative: it means there was not much collinearity left for the penalty to fix, which is a sign the feature set was already sound. If Ridge had rescued a badly-behaved Linear Regression, that would have pointed at a problem in the features, not a virtue in Ridge.

6.3 Settling the location question with an experiment

Section 4.5 argued that the published locality numbers beat one-hot encoding thousands of locality names. An argument is not evidence, so we ran all three options. The locality one-hot encoder was placed *inside* the pipeline, so the list of localities big enough to keep was rebuilt on each fold — the difference between a clean experiment and a leaky one.

Approach	Model	R-squared (log)	MAE (lakh)
locality numbers only (current)	Ridge	0.633	76.7
locality numbers only (current)	Random Forest	0.772	52.9
locality numbers + one-hot	Ridge	0.646	74.6
locality numbers + one-hot	Random Forest	0.772	52.9
one-hot only, no locality numbers	Ridge	0.630	77.5
one-hot only, no locality numbers	Random Forest	0.765	54.5

The published numbers win. Adding a hundred-plus locality columns on top of them changes almost nothing, and replacing them with one-hot encoding alone is clearly worse. **One well-chosen numeric column beat a hundred indicator columns** — and unlike a one-hot column, it keeps working for a locality the model has never seen.

6.4 Tuning

Small, deliberate grids. Each setting is there because it controls something we had a reason to worry about, searched by cross-validation on the training set only.

Model	Settings chosen
Ridge Regression	alpha = 10.0

Model	Settings chosen
Random Forest	max_depth = None, min_samples_leaf = 2
Gradient Boosting	learning_rate = 0.1, max_depth = 5, n_estimators = 600
Linear Regression	(nothing to tune) =

7. Results on the held-out test set

3,868 listings, never used to make a single decision. Each tuned model was refitted on the full training set and scored once.

Model	MAE (lakh)	RMSE (lakh)	R-squared (log)	Typical error %
Baseline (median)	107.6	219.8	-0.000	40.0%
Linear Regression	77.0	215.0	0.615	24.5%
Ridge Regression	77.0	215.3	0.616	24.5%
Random Forest	54.3	143.0	0.767	15.8%
Gradient Boosting	**53.4**	**140.1**	**0.776**	**16.3%**

All four are reported, not just the winner. Showing only the best model hides how much of the performance came from the choice of model rather than from the data — and that is the first thing a reviewer will ask about.

The two tree models are close enough that **which one wins depends on the metric you care about**, and that is worth saying out loud rather than hiding behind a single headline. Gradient Boosting is ahead on MAE (53.4 vs 54.3 lakh), on RMSE and on R-squared, so it is the better model on average and for avoiding large misses. Random Forest is ahead on typical percentage error (15.8% vs 16.3%), which means it is slightly better on the cheaper listings where a given rupee error hurts proportionally more. Gradient Boosting is selected here because MAE was declared the headline metric before the models were run; if the goal were uniform percentage accuracy across price bands, Random Forest would be the defensible pick.

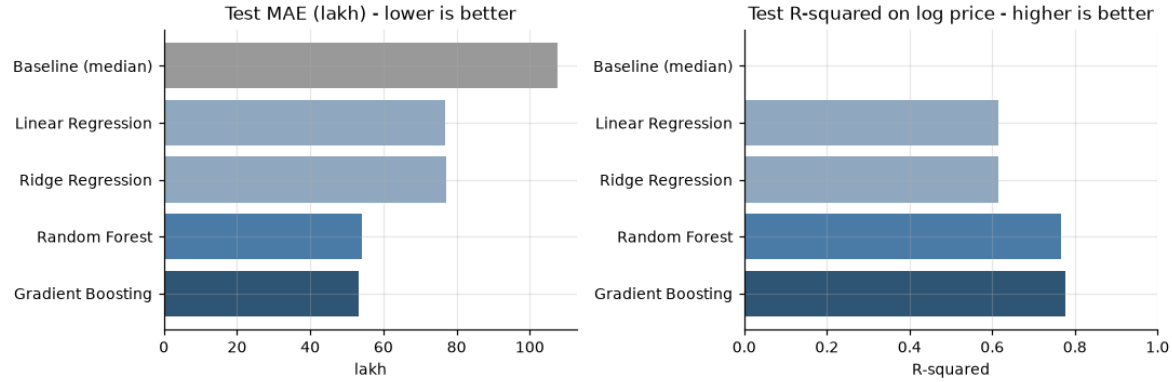


Figure 5. Test-set performance. The grey bar is the 'always guess the median' baseline.

The test numbers sit close to the cross-validation numbers. That agreement is itself a result: it means nothing was accidentally tuned against the test set, and the protocol held.

7.1 Where the model goes wrong

A single average hides the shape of the mistakes, so two diagnostics matter more here than any further tuning would.

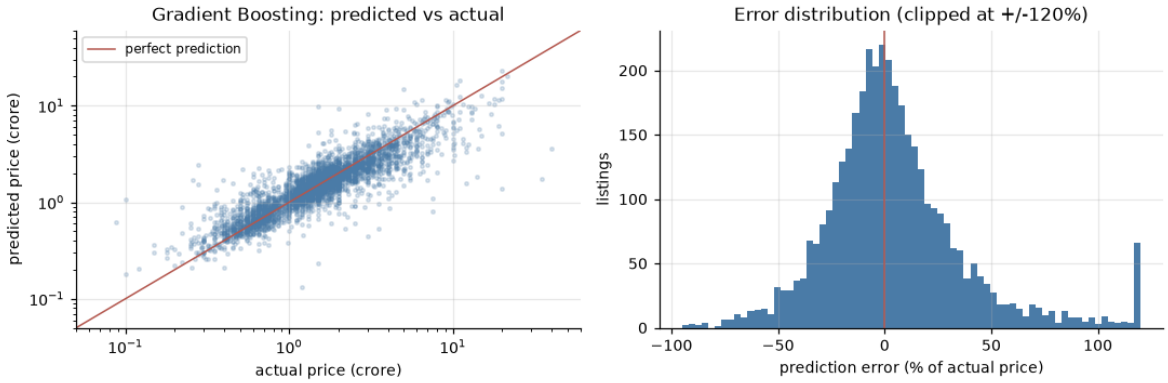


Figure 6. Left: predicted against actual price, both on log scales; the red line is a perfect prediction. Right: how far off the predictions are, as a percentage of the true price.

The error histogram is roughly centred on zero, which is the first thing to check — a model that was systematically high or low everywhere would point to a bug, typically forgetting to undo the log transform. **34% of predictions land within 10%** of the true asking price and **66% within 25%**.

Price band	Listings	Typical actual price	MAE (lakh)	Typical error %
under 50L	289	Rs 40 lakh	17.0	26.2%
50L-1Cr	783	Rs 70 lakh	18.8	16.0%
1-2Cr	1,738	Rs 150 lakh	30.0	13.7%
2-5Cr	848	Rs 295 lakh	76.1	18.2%
above 5Cr	210	Rs 700 lakh	333.3	32.7%

The pattern is consistent and has one cause: **thin data at the edges of the market**. The scrape was organised by price band, so the cheapest and most expensive segments are covered sparsely, and expensive property is also the most individual — a Rs 20 crore villa's price depends on finishes, plot shape and view, none of which appear in 23 columns.

7.2 What the model actually relies on

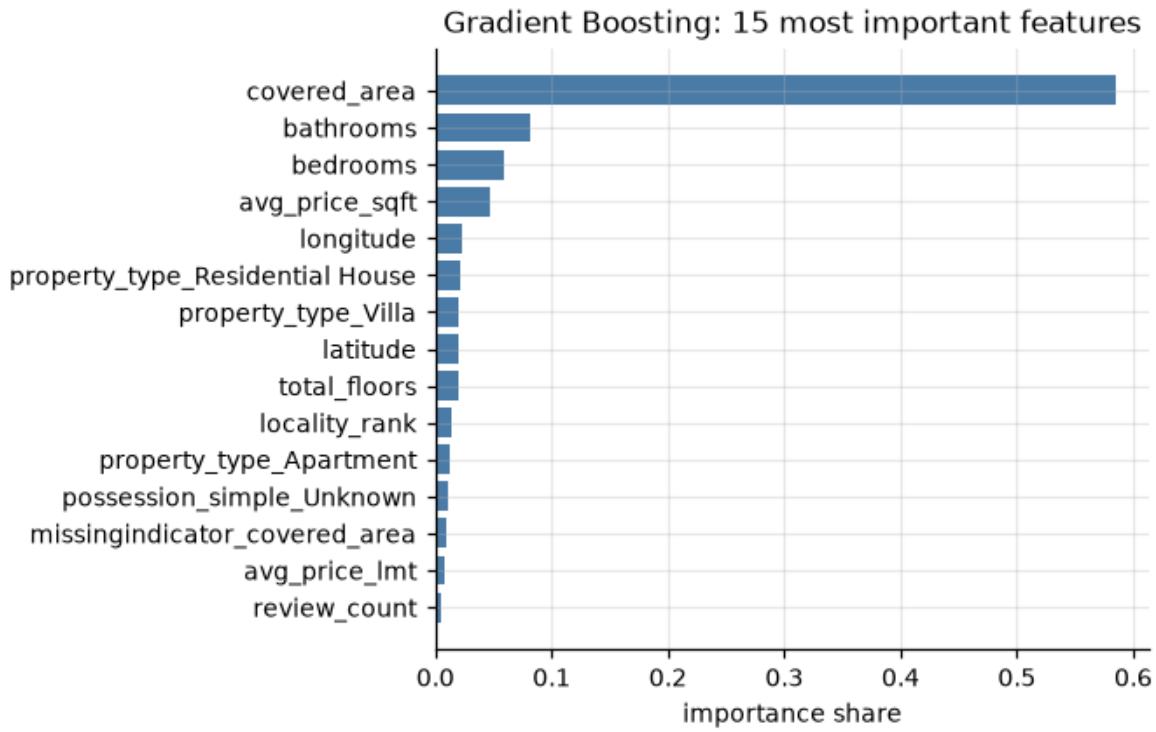


Figure 7. The features the winning model leans on most.

Feature	Share of the model's decisions
covered_area	58.5%
bathrooms	8.1%
bedrooms	5.9%
avg_price_sqft	4.7%
longitude	2.3%
property_type_Residential House	2.1%
property_type_Villa	2.0%
latitude	2.0%

Covered area dominates, the locality price aggregates come next, then bathroom and bedroom counts. That ordering is a sanity check in its own right: it is what any property valuer would tell you matters. When a feature you cannot explain tops this chart, the usual cause is leakage — so a boring, sensible ranking here is good news.

8. Honest limitations

- **It predicts asking prices, not sale prices.** That is what was scraped. Sellers ask optimistically, so this is a model of what people advertise, not of what property is worth.
- **Percentage error is worse at both ends of the market** (section 7.1), because the price-band scrape covers them thinly.
- **Validation is mildly optimistic** wherever the locality average price is used. The website computed that average over a pool of listings that overlaps ours, which the cleaning report flagged and which has not gone away. It is most pronounced in localities with few listings behind the average.

- **Coordinates are locality-resolution at best.** Many listings inherited their locality's centre point during cleaning, so latitude and longitude are coarser than they look.
- **The price mix is an artefact of the scrape**, so nothing here supports a claim about the Hyderabad market overall — only about the relationship between features and price.

9. What we would do next

In the order most likely to pay off:

- **Parse the amenity columns.** Turn amenities and flooring type into simple yes/no columns. They are currently set aside only because they need string splitting, and they are present on richer listings where the model is weakest.
- **Try HistGradientBoosting or XGBoost.** Both handle missing values natively, train much faster than plain Gradient Boosting, and usually score better.
- **Add target encoding for locality** — the average price of training listings per locality — computed strictly inside each cross-validation fold. Done carelessly it is leakage; done correctly it is often the single biggest gain available here.
- **Only then look at the description text**, and only after stripping the price out of it.
- **Model the cheap and expensive bands separately**, or weight them, if the goal is uniform percentage accuracy rather than the best average.

10. The four things worth explaining in an interview

A log transform was worth more than any feature

Skew 5.82 to 0.09, one line of code. The linear models were unusable without it. It is a good demonstration that understanding the target matters more than adding features — the first question about any regression target should be whether its distribution suits the loss function.

The leak was found by arithmetic, not by intuition

price_per_sqft_true looks like a feature and ranks like the best one. Dividing price by area and comparing gave exactly 1.0 on all 15,472 rows, which settles it. The test that generalises: *would I know this value before I knew the price?* It is also why the locality averages were kept — they pass that same test, and over-pruning out of caution costs accuracy just as surely as leakage does.

One good number beat a hundred dummy columns

2,177 localities one-hot encoded lost to four published locality statistics, and the difference was measured, not assumed. High cardinality is not automatically a reason for one-hot encoding; the question is whether a compact numeric summary of the same thing already exists.

A missing value can be a feature

The floor number is absent for 99.9% of villas, because the question is meaningless for them. Filling the gap and adding a marker column lets the model use the absence. “Missing” is not always an error to be patched over — sometimes it is the data telling you something.

Appendix: what was produced, and how to re-run it

File	What it is
notebooks/03-eda.ipynb	the exploration, six questions, with the figures
notebooks/04-feature-engineering-and-models.ipynb	features, the four models, tuning, final evaluation
data/processed/train.csv, test.csv	the 80/20 split, fixed seed 42
models/gradient_boosting.joblib	the winning model, with its preprocessing baked in
reports/model_results.json	every number in this report, machine-readable
reports/figures/	the seven figures
reports/make_report.py	the script that builds this PDF from that JSON

Run notebook 03 and then notebook 04 from a fresh kernel, then `python reports/make_report.py`. The random seed is fixed at 42 throughout, so the split, the folds and the models reproduce exactly. Because the saved model carries its own preprocessing, predicting on a new listing is one call — no chance of forgetting a transformation.

Phases 2 and 3 of 4. Next: either the improvements in section 9, or deployment of the saved model behind a small prediction interface.